

BDCR: Logic to Go From Description → ERD (Step-by-step)

Step 1) Read the story and underline NOUNS (things you store many of)

Rule: if it's a "thing" you'll have lots of records for, it's probably a table.

From the BDCR description, the nouns that become tables are:

- Company
- Customer
- Reservation
- Rental
- Bill
- Bill Detail (because a bill contains many rentals)
- Employee
- Job Type
- Vehicle Type
- Vehicle
- Maintenance (maintenance log entry)
- Maintenance Detail (line items for work done)
- Part
- Part Type
- Part Used (because many parts can be used in many maintenance detail lines)

Step 2) Underline VERBS (actions between nouns)

Rule: verbs usually become relationships (lines) between tables.

Key verbs in the story:

- Company **employs** Customer
- Customer **reserves** (creates) Reservation
- Customer **rents** Vehicle (this event is Rental)
- Employee **signs out** a Rental (start employee)
- Employee **signs in** a Rental (end employee)
- Company/Customer **is billed** (Bill)
- Bill **contains** Rentals (Bill Detail)
- Vehicle **requires** Maintenance
- Employee **performs** Maintenance / Maintenance Detail
- Maintenance **includes** Maintenance Detail lines
- Maintenance Detail **uses** Parts (Part Used bridge)

Step 3) Turn “events” into tables (when the story describes a form or transaction)

Rule: if the story describes a form with its own fields/dates, it's usually an **event table**.

Events in BDCR:

- Reservation (reservation details)
- Rental (checkout + trip completion info)
- Bill (monthly billing)

- Maintenance (maintenance log entry)
- Maintenance Detail (the line items of the maintenance work)
- Part Used (parts usage record)

Step 4) Decide the “one vs many” for each relationship

Beginner rules:

- If one A can have many B → **1-to-many** (FK goes on the “many” side)
- If many A can connect to many B → **many-to-many** → you need a **bridge table**

Where BDCR becomes many-to-many:

- **Bill** ↔ **Rental** is many-to-many → bridge: `bill_detail`
- **MaintenanceDetail** ↔ **Part** is many-to-many → bridge: `part_used`

Step 5) Add keys (PKs) and put FKs where the logic says they belong

Rule:

- Each table gets a **primary key** (unique id)
- Each 1-to-many relationship places the **foreign key** in the “many” table
- Each bridge table stores both parent keys (usually as a composite PK)

Final ERD Contents (Tables + Columns)

Below is the complete set of entities (tables) and attributes that appear on the ERD.

company

- **PK** companyCode
- companyName

customer

- **PK** customerId
- **FK** companyCode → company.companyCode
- customerName

reservation

- **PK** reservationId
- **FK** customerId → customer.customerId
- **FK** vehicleTypeId → vehicle_type.vehicleTypeId
- expectedStartDate
- destination

vehicle_type

- **PK** vehicleTypeId
- vehicleTypeName
- mileageRate

vehicle

- **PK** vehicleId
- **FK** vehicleTypeId → vehicle_type.vehicleTypeId

employee

- **PK** employeeId
- employeeName
- jobId (shown on ERD as a field; relationship is implied via job_type)
- inspectionFlag

job_type

- **PK** jobId
- jobTypeName

rental

- **PK** rentalId
- **FK** reservationId → reservation.reservationId
- **FK** customerId → customer.customerId
- **FK** startEmployeeId → employee.employeeId
- **FK** endEmployeeId → employee.employeeId
- **FK** vehicleId → vehicle.vehicleId
- startDate
- endDate

- odometerStart
- odometerEnd
- gallonsFuel

bill

- **PK** billId
- **FK** customerCode → company.companyCode (*note: ERD labels it customerCode but it points to companyCode conceptually*)
- month

bill_detail (bridge table: bill ↔ rental)

- **PK/FK** billId → bill.billId
- **PK/FK** rentalId → rental.rentalId
- month

maintenance

- **PK** maintenancelId
- **FK** employeeId → employee.employeeId
- **FK** vehicleId → vehicle.vehicleId
- maintenanceNeeded
- startDate
- endDate

maintenanceDetail

- **PK** detailId
- **FK** employeeId → employee.employeeId
- **FK** maintenancelId → maintenance.maintenancelId

part_type

- **PK** partTypeId
- partTypeName

part

- **PK** partId
- **FK** partTypeId → part_type.partTypeId
- partName
- numberInInventory
- minimumQuantity

part_used (bridge table: part ↔ maintenanceDetail)

- **PK/FK** partId → part.partId
- **PK/FK** detailId → maintenanceDetail.detailId
- **FK** employeeId → employee.employeeId

Relationship List (What the Lines Mean)

This is the “readable ERD” version students can map to the diagram.

- company **employs** customer (1 company → many customers)
- customer **reserves** reservation (1 customer → many reservations)
- reservation **requests** vehicle_type (many reservations → 1 vehicle type)
- vehicle_type **includes** vehicle (1 vehicle type → many vehicles)
- customer **rents** rental (1 customer → many rentals)
- reservation **leads to** rental (1 reservation → 0/1 or many rentals depending on business rules; ERD stores reservationId in rental)
- rental **uses** vehicle (many rentals → 1 vehicle)
- rental **is signed out by** employee (many rentals → 1 start employee)
- rental **is signed in by** employee (many rentals → 1 end employee)
- company **is billed** via bill (1 company → many bills over time)
- bill **contains** rentals via bill_detail (bill ↔ rental is many-to-many)
- vehicle **requires** maintenance (1 vehicle → many maintenance entries)
- maintenance **includes** maintenanceDetail (1 maintenance → many detail lines)
- employee **performs** maintenanceDetail (1 employee → many detail lines)
- maintenanceDetail **uses** parts via part_used (maintenanceDetail ↔ part is many-to-many)
- part **belongs to** part_type (many parts → 1 part type)
- employee **records/associates** part_used (many part_used rows → 1 employee)